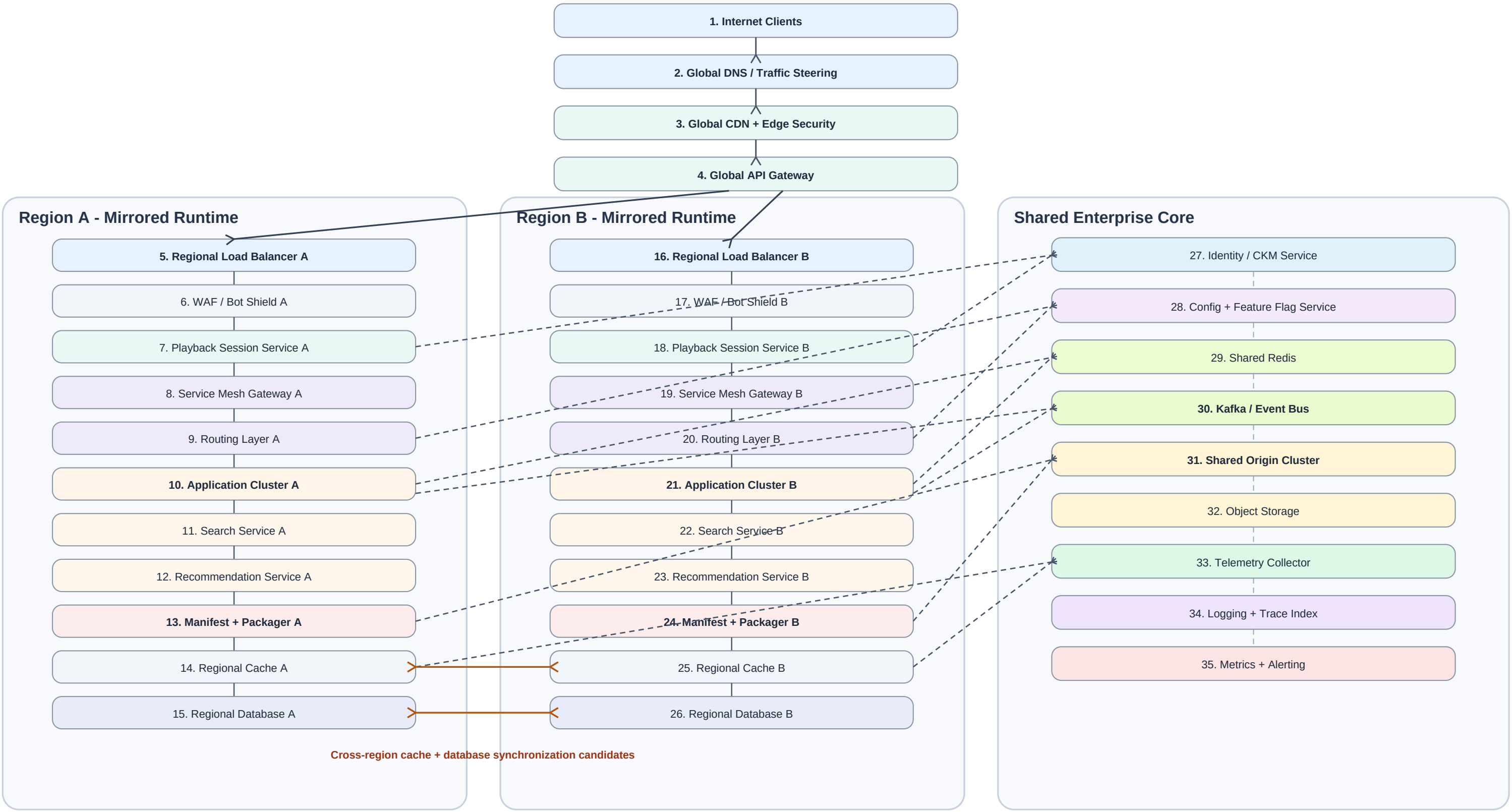




Expected journeys

1. Primary request	Client -> Global DNS -> Edge Security -> Global Gateway -> Region A/B -> Session -> Mesh -> Routing -> App Cluster
2. Authentication	Playback Session -> Identity / CKM -> token validation -> Playback Session
3. Search and recommendation	Application Cluster -> Search / Recommendation -> Redis / Event Bus
4. Content delivery	Manifest + Packager -> Shared Origin -> Object Storage -> Regional Cache -> CDN -> Client
5. Control and configuration	Config + Feature Flag -> Routing / App Cluster / Mesh
6. Event and async processing	Application Cluster -> Kafka / Event Bus -> downstream consumers
7. State synchronization	Regional Cache A <-> B; Regional Database A <-> B
8. Observability	Regional services -> Telemetry -> Logging / Trace Index -> Metrics + Alerting

Expected topology candidates

mirrored_topology Region A and Region B have highly similar structural signatures.
shared_infrastructure_topology Identity, Config, Redis, Kafka, Origin, Object Storage, and observability are shared.
fan_out Global API Gateway distributes traffic into both regional runtimes.
fan_in Telemetry from both regions converges on one shared observability chain.
cross_unit_connected_topology Explicit cache/database synchronization links connect deployment units.

Safety expectations

- All classifications remain candidate-only.
- Mirrored topology must not become an active-active claim.
- Synchronization links must not become a failover or DR claim.
- Shared infrastructure requires graph evidence from two or more deployment units.
- Traversal must remain unchanged.

Component pattern	Expected role	Why it exists
Internet Clients	external_interface	External request origin.
Global DNS / Traffic Steering	entry / control	Selects a regional destination.
Global CDN + Edge Security	entry / delivery	Edge delivery and coarse request protection.
Global API Gateway	entry / control	Global API ingress and policy boundary.
Regional Load Balancer	entry / control	Distributes requests inside a region.
WAF / Bot Shield	validation	Applies request screening.
Playback Session Service	processing / state	Creates and validates session context.
Service Mesh Gateway	control	Controls service-to-service entry.
Routing Layer	control	Chooses the regional application destination.
Application Cluster	processing	Runs core business logic.
Search Service	processing	Performs search-oriented workload.
Recommendation Service	processing	Produces recommendation responses.

Component pattern	Expected role	Why it exists
Manifest + Packager	processing / delivery	Builds playable delivery artifacts.
Regional Cache	state / delivery	Stores hot regional responses.
Regional Database	state	Persistent regional system of record.
Identity / CKM Service	auth / validation	Shared identity and key-policy validation.
Config + Feature Flag	control	Shared configuration and feature control.
Shared Redis	state	Shared low-latency state dependency.
Kafka / Event Bus	messaging	Shared asynchronous event backbone.
Shared Origin Cluster	delivery / state	Shared downstream origin dependency.
Object Storage	state / delivery	Durable shared object store.
Telemetry Collector	observability	Receives metrics, traces, and logs.
Logging + Trace Index	observability	Indexes diagnostic events and traces.
Metrics + Alerting	observability	Aggregates health signals and alerts.

Instance rule

Regional components appear twice as A/B instances; shared components appear once and must be connected from both deployment units.
Expected total architecture component instances on page 1: 35.

1. Deployment boundaries	Region A and Region B discovered as explicit deployment units.
2. Deployment membership	All A/B regional components remain inside the correct unit.
3. Replica detection	Region A and Region B produce mirrored_topology_candidate with high confidence.
4. Shared infrastructure	Graph-backed candidates include Identity/CKM, Config, Redis, Kafka, Origin, Object Storage, and Telemetry.
5. Fan-out	Global API Gateway or global entry layer fans out toward both deployment units.
6. Fan-in	Telemetry from both regions converges on shared observability.
7. Cross-unit relationships	Cache and database synchronization links are recognized without changing traversal.
8. Pattern classification	mirrored_topology plus shared infrastructure / fan-in / fan-out candidates.
9. Validator	health.valid = true; zero high-severity violations; candidate-only safety preserved.
10. Learning engine	learning-chapters.health.valid = true; no duplicate or unassigned focuses.
11. Rendering	Architecture remains readable at overview scale and supports semantic zoom to each region and shared core.
12. Narration	Mentor explains responsibilities and relationships without inventing failover, DR, or active-active behavior.

Demo safety rules

All names, components, relationships, and behaviors are fictional and derived from common enterprise architecture patterns.

No real company, partner, internal service, IP, host, log, schema, metric, or incident data is included.